

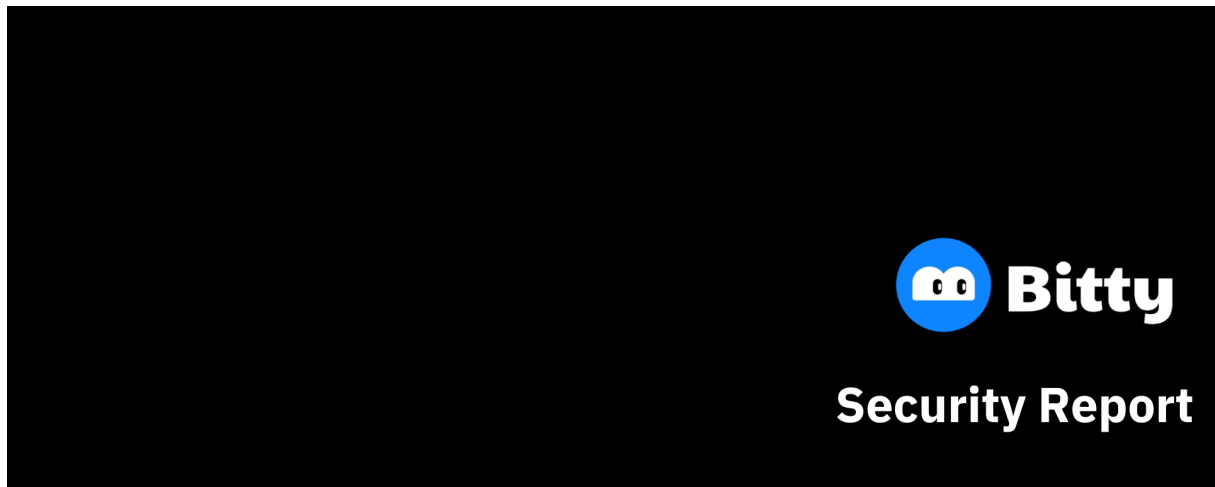
BITTY:  
SECURITY REVIEW REPORT

|  |   |   |
|--|---|---|
|  | ± | * |
|  | ± | * |
|  | ± | * |

# Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                         | <b>3</b>  |
| <b>2</b> | <b>About Verilog Solutions</b>              | <b>4</b>  |
| <b>3</b> | <b>Service Scope</b>                        | <b>5</b>  |
| 3.1      | Service Stages . . . . .                    | 5         |
| 3.2      | Methodology . . . . .                       | 5         |
| 3.3      | Audit Scope . . . . .                       | 5         |
| <b>4</b> | <b>Project Summary</b>                      | <b>6</b>  |
| <b>5</b> | <b>Findings and Improvement Suggestions</b> | <b>7</b>  |
| 5.1      | High . . . . .                              | 7         |
| 5.2      | Medium . . . . .                            | 11        |
| 5.3      | Low . . . . .                               | 14        |
| 5.4      | Informational . . . . .                     | 19        |
| <b>6</b> | <b>Appendix</b>                             | <b>21</b> |
| 6.1      | Appendix I: Severity Categories . . . . .   | 21        |
| 6.2      | Appendix II: Status Categories . . . . .    | 21        |
| <b>7</b> | <b>Disclaimer</b>                           | <b>22</b> |

## 1 | Introduction



**Figure 1.1:** BITTY Report Cover

This report presents our engineering engagement with the BITTY team on their ETHEREUM smart contracts security analysis.

## 2 | About Verilog Solutions

Founded by a group of cryptography researchers and smart contract engineers in North America, Verilog Solutions elevates the security standards for Web3 ecosystems by being a full-stack Web3 security firm covering smart contract security, consensus security, and operational security for Web3 projects.

Verilog Solutions team works closely with major ecosystems and Web3 projects and applies a quality above quantity approach with a continuous security model. Verilog Solutions onboards the best and most innovative projects and provides the best-in-class advisory services on security needs, including on-chain and off-chain components.

## 3 | Service Scope

### 3.1 | Service Stages

Our auditing service includes the following two stages:

- Pre-Audit Consulting Service
- Smart Contract Auditing Service

#### 3.1.1 | Pre-Audit Consulting Service

As a part of the audit service, the Verilog Solutions team worked closely with the BITTY development team to discuss potential vulnerability and smart contract development best practices in a timely fashion. Verilog Solutions team is very appreciative of establishing an efficient and effective communication channel with the BITTY team, as new findings were exchanged promptly and fixes were deployed quickly, during the preliminary report stage.

#### 3.1.2 | Smart Contract Auditing Service

The Verilog Solutions team analyzed the entire project using a detailed-oriented approach to capture the fundamental logic and suggested improvements to the existing code. Details can be found under Findings And Improvement Suggestions.

### 3.2 | Methodology

#### ■ Code Assessment

- We evaluate the overall quality of the code and comments as well as the architecture of the repository.
- We help the project dev team improve the overall quality of the repository by providing suggestions on refactorization to follow the best practices of Web3 software engineering.

#### ■ Code Logic Analysis

- We dive into the data structures and algorithms in the repository and provide suggestions to improve the data structures and algorithms for the lower time and space complexities.
- We analyze the hierarchy among multiple modules and the relations among the source code files in the repository and provide suggestions to improve the code architecture with better readability, reusability, and extensibility.

### 3.3 | Audit Scope

Our auditing for BITTY covered the following smart contract repositories:

#### ■ bitty-eth-oracle

- link: <https://github.com/BittyIO/bitty-eth-oracle/commit/92cdbcced06b8aa140ddd1c13f9954e2ce8c47ea>

#### ■ lending-contracts

- link: <https://github.com/BittyIO/lending-contracts/commit/cef722dbaeb74da7855a5c1deb3165ad9c884496>

#### ■ WrappedERC721

- link: <https://github.com/BittyIO/WrappedERC721/commit/e8d62220c14b716f375559c5cf22be292a28172b>

## 4 | Project Summary

Bitty is a peer-to-pool lending protocol on Ethereum and Bitcoin layer 1, with the NFT and Crypto assets both supported. The main code structure and audit scope contains the following repositories:

### ■ bitty-eth-oracle

- A decentralized NFT price oracle service that provides real-time floor price data for NFT collections to on-chain smart contracts. This service aggregates price data from multiple NFT marketplaces.
- **link:** <https://github.com/BittyIO/bitty-eth-oracle>

### ■ lending-contracts

- This repository contains the smart contracts source code and markets configuration for bitty Lending Protocol. The repository uses Hardhat as development environment for compilation, testing and deployment tasks.
- **link:** <https://github.com/BittyIO/lending-contracts>

### ■ WrappedERC721

- Standardizing the NFT contract while enabling delegation capabilities, maintainin,g secure asset custody in vaults, and providing upgradeable infrastructure for future enhancements.
- **link:** <https://github.com/BittyIO/WrappedERC721>

## 5 | Findings and Improvement Suggestions

| Severity      | Total | Acknowledged | Resolved |
|---------------|-------|--------------|----------|
| High          | 2     | 2            | 2        |
| Medium        | 3     | 3            | 3        |
| Low           | 5     | 5            | 5        |
| Informational | 2     | 2            | 2        |

### 5.1 | High

#### 5.1.1 | Incorrect Vault Ownership Design Breaks Core Functionality and Locks NFTs and Tokens

| Severity | High                                                                                                                                                           |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source   | BittyIO/WrappedERC721/src/WrappedERC721.sol:L62-L64<br>BittyIO/WrappedERC721/src/WrappedERC721.sol:L38-L45<br>BittyIO/WrappedERC721/src/AssetVault.sol:L74-L76 |
| Commit   | e8d6222;                                                                                                                                                       |
| Status   | Resolved;                                                                                                                                                      |

#### ■ Description

The current design contains a critical vault ownership flaw, which not only leads to irreversible NFT/token lock-in but also renders key functionality (like setDelegateCashV2) unusable in many cases.

#### ■ Exploit Scenario

##### 1. Vault Ownership Locks NFTs from Other Collections

Each user is assigned a single deterministic AssetVault, but the vault is owned by the first Wrapped-ERC721 contract that creates it. This design causes a severe permission issue:

- If a user stakes NFTs from multiple WrappedERC721 collections, Only the first collection (i.e., vault owner) is allowed to withdraw NFTs. Other collections fail the onlyOwner check, and their NFTs become permanently locked.

Impact: Users unknowingly lose access to staked NFTs from all but the first collection.

- A simplified PoC demonstrating this bug is provided below:

```
.....
function testWrongPermissionCheck() public {
    vm.startPrank(user);
    address assetVaultForUser = address(assetVaultRegistry.get(user));
    assertEq(assetVaultForUser, address(0x0));
    // user has no vault yet

    uint256[] memory tokenIds = new uint256[](1);
    tokenIds[0] = tokenId;

    // stake first NFT collection
    erc721s[0].setApprovalForAll(WrappedERC721s[0], true);
    WrappedERC721(WrappedERC721s[0]).mint(tokenIds);
    assetVaultForUser = address(assetVaultRegistry.get(user));
    console.log("assetVault for user", assetVaultForUser);
    assertTrue(assetVaultForUser != address(0x0));
    assertEq(erc721s[0].ownerOf(tokenId), address(assetVaultForUser));
```

```

assertEq(WrappedERC721(WrappedERC721s[0]).ownerOf(tokenId), user);
// vault owner is now stNFT1
console.log("the owner of user's assetVault", AssetVault(assetVaultForUser).owner());
assertEq(AssetVault(assetVaultForUser).owner(), address(WrappedERC721s[0]));

// stake second NFT collection
erc721s[1].setApprovalForAll(WrappedERC721s[1], true);
WrappedERC721(WrappedERC721s[1]).mint(tokenIds);
assetVaultForUser = address(assetVaultRegistry.get(user));
console.log("assetVault for user", assetVaultForUser);
assertEq(erc721s[1].ownerOf(tokenId), address(assetVaultForUser));
assertEq(WrappedERC721(WrappedERC721s[1]).ownerOf(tokenId), user);

// burn stNFT1 - works
WrappedERC721(WrappedERC721s[0]).burn(tokenIds);
assertEq(erc721s[0].ownerOf(tokenId), user);

// burn stNFT2 - fails, vault owner is still stNFT1
vm.expectRevert();
WrappedERC721(WrappedERC721s[1]).burn(tokenIds);
// user can only withdraw first staked NFT

vm.stopPrank();

```

Similarly, the setDelegateCashV2 and ERC20/ERC1155 Withdrawals function in AssetVault is protected by onlyOwner, but only the first WrappedERC721 contract owns the vault.

#### ■ Recommendations

Update the ownership of AssetVault to AssetVaultRegistry, rather than WrappedERC721 contract. Check the full discussion of the issue here: <https://github.com/BittyIO/WrappedERC721/issues/9>

#### ■ Results

Resolved in commit 9e13ed6 and 94a231b.

### 5.1.2 | Unable to obtain the correct lowest price

| Severity | High                                         |
|----------|----------------------------------------------|
| Source   | bitty-eth-oracle/app/oracle/price.go:L42-78; |
| Commit   | 92cdbcc;                                     |
| Status   | Resolved;                                    |

#### ■ Description

The key of the map is tokenId, and the same tokenId should be the same key. However, this uses pointers, which means that even if the values they point to are the same (for example, value=16), they are still different keys. Here is a simple PoC:

```
package main

import (
    "fmt"
    "math/big"
    "testing"
)

func TestPoC(t *testing.T) {
    // 1. create a map: *big.Int -> string
    m := make(map[*big.Int]string)

    // 2. create two *big.Int variables, both are 16
    tokenId1 := big.NewInt(16)
    tokenId2 := big.NewInt(16)
    m[tokenId1] = "Price = 3 ether"
    m[tokenId2] = "Price = 4 ether"

    // 3. print the values of the map
    fmt.Println("m[tokenId1]:", m[tokenId1])
    fmt.Println("m[tokenId2]:", m[tokenId2])
    fmt.Println("tokenId1 == tokenId2:", tokenId1 == tokenId2)

    fmt.Println("m[16]:", m[big.NewInt(16)])

    /* result:
       m[tokenId1]: Price = 3 ether
       m[tokenId2]: Price = 4 ether
       tokenId1 == tokenId2: false
       m[16]:      <----- not found
    */
}
```

If there were old values before, no comparison was made to find the lowest price.

```
// for the same seller, pick the lowest price
sellerMap := make(map[common.Address]*TokenFloorPrice)
or _, price := range prices {
    if _, ok := sellerMap[price.Seller]; !ok {
        sellerMap[price.Seller] = price
    } else {
        if price.Price.Cmp(sellerMap[price.Seller].Price) < 0 {
            sellerMap[price.Seller] = price
        } // (*) <-----
```

```

}

rices = lo.Values(sellerMap)

/ for the same token id, pick the lowest price
okenIdMap := make(map[*big.Int]*TokenFloorPrice)
or _, price := range prices {
if _, ok := tokenIdMap[price.TokenId]; !ok {
    tokenIdMap[price.TokenId] = price
} // <----- audit: We should choose the lowest price in the same way as mentioned above (*).

rices = lo.Values(tokenIdMap)

```

#### ■ Exploit Scenario

Check the above explanation.

#### ■ Recommendations

1. Do not use pointer types as keys for the mapping.
2. Add logic for the same token id, refer to the (\*) method.

#### ■ Results

Resolved in commit 84b3e63.

## 5.2 | Medium

### 5.2.1 | ETH Trapped inside UpgradeableProxy

| Severity | Medium                                      |
|----------|---------------------------------------------|
| Source   | WrappedERC721/src/UpgradeableProxy.sol:L20; |
| Commit   | e8d6222;                                    |
| Status   | Resolved;                                   |

#### ■ Description

The proxy contract defines a `receive()` function, so it accepts ETH sent directly or via empty-calldata calls to the proxy contract address. However, none of the implementation contracts (WrappedERC721, AssetVault, AssetVaultRegistry, WrappedERC721Registry, etc.) provides a function to withdraw ETH from the contract. Thus, the `receive()` function can be removed.

#### ■ Exploit Scenario

The ETH can only be locked when users accidentally send the ETH to the proxy contract.

#### ■ Recommendations

Add relevant logic to withdrawal the locked ETH.

#### ■ Results

Resolved in commit 3d2cab3.

### 5.2.2 | Replace hard-coded context.Background()/TODO() with the passed-in ctx

| Severity | Medium                                       |
|----------|----------------------------------------------|
| Source   | bitty-eth-oracle/config/provider_aws.go:L26; |
| Commit   | 92cdbcc;                                     |
| Status   | Resolved;                                    |

#### ■ Description

In ProviderAWS.Load, the library should use the ctx passed in by callers rather than calling context.Background() or context.TODO(). Hard-coded root contexts belong only at application entry points (e.g. main() or HTTP handlers).

#### ■ Exploit Scenario

N/A.

#### ■ Recommendations

Using the passed-in ctx ensures upstream cancellations, deadlines, and values propagate correctly.

#### ■ Results

Resolved in commit 92cdbcc.

### 5.2.3 | The CalculatePercentage() function is not rigorous enough

| Severity | Medium                                          |
|----------|-------------------------------------------------|
| Source   | bitty-eth-oracle/config/provider_aws.go:L40-63; |
| Commit   | 92cdbcc;                                        |
| Status   | Resolved;                                       |

#### ■ Description

When price1 is 0 and price2 is not 0, the result should be infinity in mathematical terms. And using 1.0 here is inappropriate because the result could generally be 1.0. A special value should be used to distinguish this situation.

#### ■ Exploit Scenario

```
.....
// Calculate percentage change of price2 relative to price1
// Returns positive number if price2 > price1, negative if price2 < price1
func CalculatePercentage(price1 *big.Int, price2 *big.Int) float64 {
    if price1 == nil || price2 == nil {
        return 0.0
    }

    // Convert to decimal for precise calculation
    price1Decimal := decimal.NewFromBigInt(price1, 0)
    price2Decimal := decimal.NewFromBigInt(price2, 0)

    if price1Decimal.IsZero() {
        if price2Decimal.IsZero() {
            return 0.0
        }
        return 1.0 // <---- audit: should not be 1.0.
                  //      could be a special value, e.g. math.Inf(1)
    }

    // Percentage change = (price2 - price1) / price1
    diff := price2Decimal.Sub(price1Decimal)
    percentage := diff.Div(price1Decimal)
    result, _ := percentage.Float64()
    return result
}
```

#### ■ Recommendations

A special value should be used to distinguish this situation.

#### ■ Results

Resolved in commit 92cdbcc.

## 5.3 | Low

### 5.3.1 | setBaseURI() function in WrappedERC721 contract doesn't work

| Severity | Low                                           |
|----------|-----------------------------------------------|
| Source   | WrappedERC721/src/WrappedERC721.sol:L102-104; |
| Commit   | e8d6222;                                      |
| Status   | Resolved;                                     |

#### ■ Description

In production, the WrappedERC721 contract should be created and registered in the WrappedERC721Registry contract. The owner of the WrappedERC721 contract is WrappedERC721Registry. However, the WrappedERC721Registry contract lacks a setBaseURI() function, so nobody can call WrappedERC721.setBaseURI(string).

#### ■ Exploit Scenario

Function 'setBaseURI()' cannot be triggered by the current code structure.

#### ■ Recommendations

Add related logic in WrappedERC721Registry contract, or fine tune the ownership.

#### ■ Results

Resolved in commit bf5771e.

### 5.3.2 | Parameter 'tokenIdToAssetVault' not cleared after burn, may cause confusion

| Severity | Low                                         |
|----------|---------------------------------------------|
| Source   | WrappedERC721/src/WrappedERC721.sol:L75-83; |
| Commit   | e8d6222;                                    |
| Status   | Resolved;                                   |

#### ■ Description

In WrappedERC721.sol, after a WrappedERC721 token is burned, the corresponding entry in the tokenIdToAssetVault mapping is not deleted:

```
tokenIdToAssetVault[tokenId]; // ← still exists after burn
```

#### ■ Exploit Scenario

- Incorrect lookups for burned tokenIds.
- Misleading frontend or off-chain logic, which may assume the NFT is still associated with a vault or active.
- Potential bugs in future extensions that depend on this mapping to determine active tokens or vault associations.

#### ■ Recommendations

Adding logic for removing mapping.

#### ■ Results

Resolved in commit 9e13ed6.

### 5.3.3 | Initialization is incomplete

| Severity | Low                                                 |
|----------|-----------------------------------------------------|
| Source   | WrappedERC721/src/WrappedERC721Registry.sol:L24-27; |
| Commit   | e8d6222;                                            |
| Status   | Resolved;                                           |

#### ■ Description

The existing code lacks Reentrancy Guard initialization.

#### ■ Exploit Scenario

N/A.

#### ■ Recommendations

Adding related logic.

#### ■ Results

Resolved in commit 9e13ed6.

### 5.3.4 | The hardcoded content in contractURI() makes the result incorrect when deploying on non-ethereum blockchain

| Severity | Low                                       |
|----------|-------------------------------------------|
| Source   | WrappedERC721/src/WrappedERC721.sol:L125; |
| Commit   | e8d6222;                                  |
| Status   | Resolved;                                 |

#### ■ Description

It is recommended not to hardcode the ETH.

#### ■ Exploit Scenario

As the Bitty team is expecting to deploy contracts to multiple networks, this hardcode should be removed.

#### ■ Recommendations

Adjust the parameter setting to fit the design requirements.

#### ■ Results

Resolved in commit ec6adf2.

### 5.3.5 | Unused Parameters in NewMultiCallerFactory()

| Severity | Low                                         |
|----------|---------------------------------------------|
| Source   | bitty-eth-oracle/pkg/util/multicall.go:L24; |
| Commit   | 92cdbcc;                                    |
| Status   | Resolved;                                   |

#### ■ Description

```
func NewMultiCallerFactory(ctx context.Context, client *ethclient.Client, address string)
(*MultiCallerFactory, error) {
```

The parameter ctx is not used in the function NewMultiCallerFactory().

#### ■ Exploit Scenario

N/A.

#### ■ Recommendations

Either remove this parameter or use it in business logic.

#### ■ Results

Resolved in commit 3692870.

## 5.4 | Informational

### 5.4.1 | When burning WrappedERC721, it is recommended to add a recipient parameter

| Severity | Informational                            |
|----------|------------------------------------------|
| Source   | WrappedERC721/src/WrappedERC721.sol:L85; |
| Commit   | e8d6222;                                 |
| Status   | Resolved;                                |

#### ■ Description

If the user is blacklisted by the underlying NFT between minting WrappedERC721 and burning WrappedERC721, then the burn() function will be DoS. For example:

- User-1 mints with BAYC and gets stBAYC
- User-1 is blacklisted by BAYC
- User-1 calls burn() but fails because he is in the blacklist

#### ■ Exploit Scenario

N/A.

#### ■ Recommendations

It is recommended to add a new parameter, address recipient in burn().

```
assetVault_.withdrawERC721(staker_, address(_erc721), tokenId_);  
assetVault_.withdrawERC721(recipient, address(_erc721), tokenId_);
```

#### ■ Results

Resolved in commit 3692870.

### 5.4.2 | Better GasPriceMultiplier

| Severity | Informational                          |
|----------|----------------------------------------|
| Source   | bitty-eth-oracle/config/config.go:L18; |
| Commit   | 92cdbcc;                               |
| Status   | Resolved;                              |

#### ■ Description

The GasPriceMultiplier is 1.1, which is a bit small. The default value in Foundry is 130 (<https://getfoundry.sh/forgereference/forgereference-script/#options>). The gas fluctuations on the ETH mainnet sometimes are large, 1.1 may cause a long waiting time.

#### ■ Exploit Scenario

N/A.

#### ■ Recommendations

Use 1.3 (130) for GasPriceMultiplier which is the same as Foundry does. This value should be the best default value obtained after extensive testing by the Foundry team.

#### ■ Results

Resolved in commit 84b3e63.

## 6 | Appendix

### 6.1 | Appendix I: Severity Categories

| Severity    | Description                                                                                                                                                                                                                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High        | Issues that are highly exploitable security vulnerabilities. It may cause direct loss of funds / permanent freezing of funds. All high severity issues should be resolved.                                                                                                                                   |
| Medium      | Issues that are only exploitable under some conditions or with some privileged access to the system. Users' yields/rewards/information is at risk. All medium severity issues should be resolved unless there is a clear reason not to.                                                                      |
| Low         | Issues that are low risk. Not fixing those issues will not result in the failure of the system. A fix on low severity issues is recommended but subject to the clients' decisions.                                                                                                                           |
| Information | Issues that pose no risk to the system and are related to the security best practices. Not fixing those issues will not result in the failure of the system. A fix on informational issues or adoption of those security best practices-related suggestions is recommended but subject to clients' decision. |

### 6.2 | Appendix II: Status Categories

| Severity           | Description                                                               |
|--------------------|---------------------------------------------------------------------------|
| Unresolved         | The issue is not acknowledged and not resolved.                           |
| Partially Resolved | The issue has been partially resolved                                     |
| Acknowledged       | The Finding / Suggestion is acknowledged but not fixed / not implemented. |
| Resolved           | The issue has been sufficiently resolved                                  |

## 7 | Disclaimer

Verilog Solutions receives compensation from one or more clients for performing the smart contract and auditing analysis contained in these reports. The report created is solely for Clients and published with their consent. As such, the scope of our audit is limited to a review of code, and only the code we note as being within the scope of our audit is detailed in this report. It is important to note that the Solidity code itself presents unique and unquantifiable risks since the Solidity language itself remains under current development and is subject to unknown risks and flaws. Our sole goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies. Thus, Verilog Solutions in no way claims any guarantee of security or functionality of the technology we agree to analyze.

In addition, Verilog Solutions reports do not provide any indication of the technology's proprietors, business, business model, or legal compliance. As such, reports do not provide investment advice and should not be used to make decisions about investment or involvement with any particular project. Verilog Solutions has the right to distribute the Report through other means, including via Verilog Solutions publications and other distributions. Verilog Solutions makes the reports available to parties other than the Clients (i.e., "third parties") – on its website in hopes that it can help the blockchain ecosystem develop technical best practices in this rapidly evolving area of innovation.